JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class LongAdder

java.lang.Object
    java.lang.Number
        java.util.concurrent.atomic.LongAdder

All Implemented Interfaces:

Serializable

```
public class LongAdder
extends Number
implements Serializable
```

One or more variables that together maintain an initially zero `long` sum. When updates (method `add(long)`) are contended across threads, the set of variables may grow dynamically to reduce contention. Method `sum()` (or, equivalently, `longValue()`) returns the current total combined across the variables maintaining the sum.

This class is usually preferable to `AtomicLong` when multiple threads update a common sum that is used for purposes such as collecting statistics, not for fine-grained synchronization control. Under low update contention, the two classes have similar characteristics. But under high contention, expected throughput of
this class is significantly higher, at the expense of higher space consumption.

LongAdders can be used with a `ConcurrentHashMap` to maintain a scalable frequency map (a form of histogram or multiset). For example, to add a count to a `ConcurrentHashMap<String,LongAdder> freqs`, initializing if not already present, you can use `freqs.computeIfAbsent(key, k -> new LongAdder()).increment();`

This class extends `Number`, but does *not* define methods such as `equals`, `hashCode` and `compareTo` because instances are expected to be mutated, and so are not useful as collection keys.

Since:

1.8

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`LongAdder()`	

Creates a new adder with initial sum of zero.

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
void	**add**(long x)	Adds the given value.
void	**decrement**()	Equivalent to add(-1).
double	**doubleValue**()	Returns the sum() as a double after a widening primitive conversion.
float	**floatValue**()	Returns the sum() as a float after a widening primitive conversion.
void	**increment**()	Equivalent to add(1).
int	**intValue**()	Returns the sum() as an int after a narrowing primitive conversion.
long	**longValue**()	Equivalent to sum().
void	**reset**()	Resets variables maintaining the sum to zero.
long	**sum**()	Returns the current sum.
long	**sumThenReset**()	Equivalent in effect to sum() followed by reset().
String	**toString**()	Returns the String representation of the sum().

Methods declared in class Number

byteValue, shortValue

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Constructor Details

LongAdder

```
public LongAdder()
```

Creates a new adder with initial sum of zero.

Method Details

add

```
public void add(long x)
```

Adds the given value.

Parameters:

`x` - the value to add

increment

```
public void increment()
```

Equivalent to `add(1)`.

decrement

```
public void decrement()
```

Equivalent to `add(-1)`.

sum

```
public long sum()
```

Returns the current sum. The returned value is *NOT* an atomic snapshot; invocation in the absence of concurrent updates returns an accurate result, but concurrent updates that occur while the sum is being calculated might not be incorporated.

Returns:

the sum

reset

```
public void reset()
```

Resets variables maintaining the sum to zero. This method may be a useful alternative to creating a new adder, but is only effective if there are no concurrent updates. Because this method is intrinsically racy, it should only be used when it is known that no threads are concurrently updating.

sumThenReset

```
public long sumThenReset()
```

Equivalent in effect to `sum()` followed by `reset()`. This method may apply for example during quiescent points between multithreaded computations. If there are updates concurrent with this method, the returned value is *not* guaranteed to be the final value occurring before the reset.

Returns:

the sum

toString

```
public String toString()
```

Returns the String representation of the `sum()`.

Overrides:

`toString` in class `Object`

Returns:

the String representation of the `sum()`

longValue

```
public long longValue()
```

Equivalent to `sum()`.

Specified by:

`longValue` in class `Number`

Returns:

the sum

intValue

```
public int intValue()
```

Returns the `sum()` as an `int` after a narrowing primitive conversion.

Specified by:

`intValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `int`.

floatValue

```
public float floatValue()
```

Returns the `sum()` as a `float` after a widening primitive conversion.

Specified by:

`floatValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `float`.

doubleValue

```
public double doubleValue()
```

Returns the `sum()` as a `double` after a widening primitive conversion.

Specified by:

`doubleValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `double`.

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.